

```
from google.colab import files
uploaded = files.upload()
<IPython.core.display.HTML object>
Saving SALES_FORECASTING.webp to SALES_FORECASTING (1).webp
Saving train.csv to train (1).csv

import os

print("Uploaded files:")
for file in uploaded.keys():
    print(file)

Uploaded files:
SALES_FORECASTING (1).webp
train (1).csv

from PIL import Image
import matplotlib.pyplot as plt

image_path = next(iter(uploaded.keys()))

img = Image.open(image_path)

plt.figure(figsize=(12, 6))
plt.imshow(img)
plt.axis("off")
plt.title("Sales Forecasting")
plt.show()
```

Sales Forecasting



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.statespace.sarimax import SARIMAX

from sklearn.metrics import mean_absolute_error, mean_squared_error

import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-whitegrid')
```

UPLOAD DATASET

FIRST FIVE ROWS

```
df = pd.read_csv('/content/train.csv')
df.head()
```

```

{"summary":{"\n  \"name\": \"df\", \n  \"rows\": 9800, \n  \"fields\": [\n    {\n      \"column\": \"Row ID\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 2829, \n        \"min\": 1, \n        \"max\": 9800, \n        \"num_unique_values\": 9800, \n        \"samples\": [\n          533, \n          873, \n          1150\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Order ID\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4922, \n        \"samples\": [\n          \"CA-2018-105809\", \n          \"CA-2018-144491\", \n          \"CA-2018-147564\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Order Date\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 1230, \n        \"samples\": [\n          \"21/06/2018\", \n          \"20/04/2016\", \n          \"20/05/2017\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Ship Date\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 1326, \n        \"samples\": [\n          \"26/08/2015\", \n          \"14/12/2015\", \n          \"19/04/2018\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Ship Mode\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"Standard Class\", \n          \"Same Day\", \n          \"Second Class\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Customer ID\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 793, \n        \"samples\": [\n          \"DJ-13510\", \n          \"MD-17350\", \n          \"NF-18475\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Customer Name\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 793, \n        \"samples\": [\n          \"Don Jones\", \n          \"Maribeth Dona\", \n          \"Neil Franz\\u00f6sisch\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Segment\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"Consumer\", \n          \"Corporate\", \n          \"Home Office\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"Country\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 1, \n        \"samples\": [\n          \"United States\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"City\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 529, \n        \"samples\": [\n          \"Burlington\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }\n  ]\n}

```

```

n    },\n    {\n        \"column\": \"State\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 49,\n            \"samples\": [\n                \"Delaware\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Postal Code\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 32041.22341281317, \n            \"min\": 1040.0, \n            \"max\": 99301.0, \n            \"num_unique_values\": 626, \n            \"samples\": [\n                56301.0 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Region\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 4, \n            \"samples\": [\n                \"West\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Product ID\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 1861, \n            \"samples\": [\n                \"TEC-PH-10001580\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Category\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 3, \n            \"samples\": [\n                \"Furniture\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Sub-Category\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 17, \n            \"samples\": [\n                \"Bookcases\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Product Name\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 1849, \n            \"samples\": [\n                \"Imation 32GB Pocket Pro USB 3.0 Flash Drive - 32 GB - Black - 1 P ...\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    }, \n    {\n        \"column\": \"Sales\", \n        \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 626.6518748388047, \n            \"min\": 0.444, \n            \"max\": 22638.48, \n            \"num_unique_values\": 5757, \n            \"samples\": [\n                35.34 \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        } \n    } \n], \n\"type\": \"dataframe\", \"variable_name\": \"df\"}

```

UNDERSTAND DATASET

```

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

```

Rows: 9800
 Columns: 18

```
print(df.columns.tolist())

['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
'Product Name', 'Sales']
```

MISSING VALUE

```
df.isnull().sum()
```

```
Row ID          0
Order ID        0
Order Date      0
Ship Date       0
Ship Mode       0
Customer ID     0
Customer Name   0
Segment         0
Country         0
City            0
State           0
Postal Code     11
Region          0
Product ID      0
Category        0
Sub-Category    0
Product Name    0
Sales           0
dtype: int64
```

DUPLICATE VALUE

```
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

```
df = df.drop_duplicates()
```

CONVERTING DATE COLUMN

```
df['Order Date'] = pd.to_datetime(
    df['Order Date'],
    dayfirst=True
)
```

MONTHLY SALES TIME SERIES

```
df[['Order Date', 'Sales']].head()

{"summary":{"\n  \"name\": \"df[['Order Date', 'Sales']]\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Order Date\", \n      \"properties\": {\n        \"dtype\": \"date\", \n        \"min\": \"2016-10-11 00:00:00\", \n        \"max\": \"2017-11-08 00:00:00\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"2017-11-08 00:00:00\", \n          \"2017-06-12 00:00:00\", \n          \"2016-10-11 00:00:00\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"Sales\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 427.5596264242568, \n        \"min\": 14.62, \n        \"max\": 957.5775, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          731.94, \n          22.368, \n          14.62 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    } \n  ] \n}, \"type\": \"dataframe\"}

monthly_sales = (
    df.set_index('Order Date')
      .resample('MS')['Sales']
      .sum()
)

monthly_sales.head()

Order Date
2015-01-01    14205.707
2015-02-01     4519.892
2015-03-01    55205.797
2015-04-01    27906.855
2015-05-01    23644.303
Freq: MS, Name: Sales, dtype: float64
```

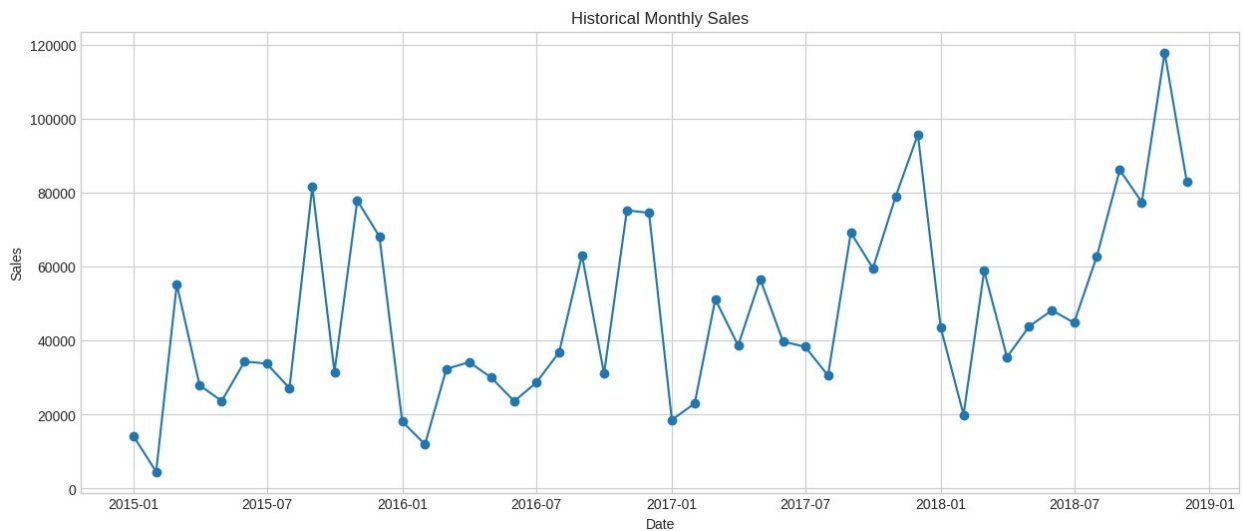
HISTORICAL MONTHLY SALES

```
plt.figure(figsize=(15,6))

plt.plot(
    monthly_sales.index,
    monthly_sales.values,
    marker='o'
)

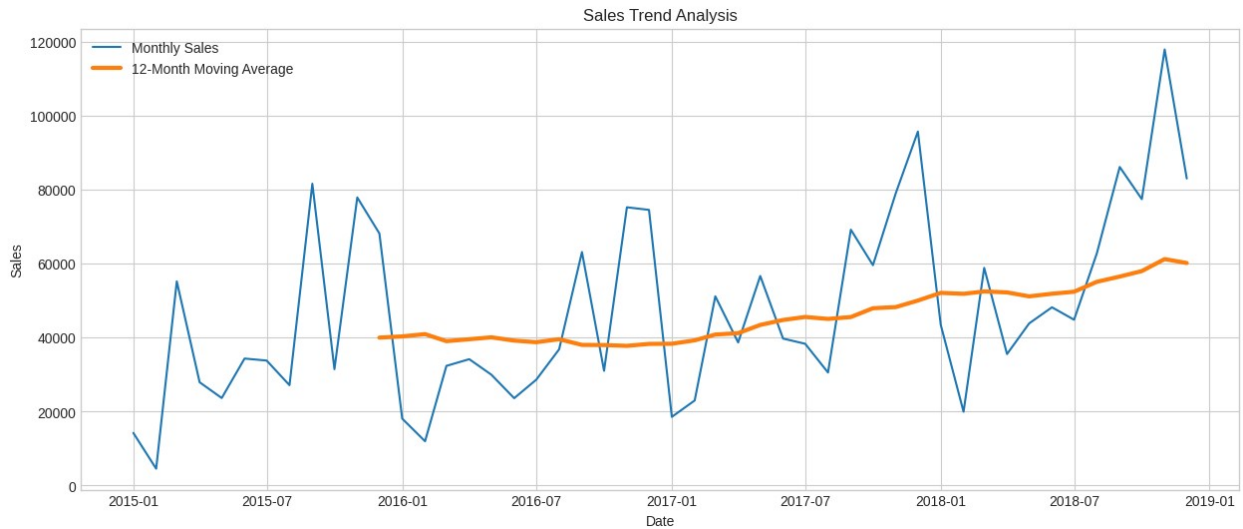
plt.title('Historical Monthly Sales')
plt.xlabel('Date')
plt.ylabel('Sales')
```

```
plt.show()
```



SALES TREND ANALYSIS

```
rolling_sales = monthly_sales.rolling(  
    window=12  
) .mean()  
  
plt.figure(figsize=(15,6))  
  
plt.plot(  
    monthly_sales,  
    label='Monthly Sales'  
)  
  
plt.plot(  
    rolling_sales,  
    label='12-Month Moving Average',  
    linewidth=3  
)  
  
plt.title('Sales Trend Analysis')  
plt.xlabel('Date')  
plt.ylabel('Sales')  
  
plt.legend()  
  
plt.show()
```

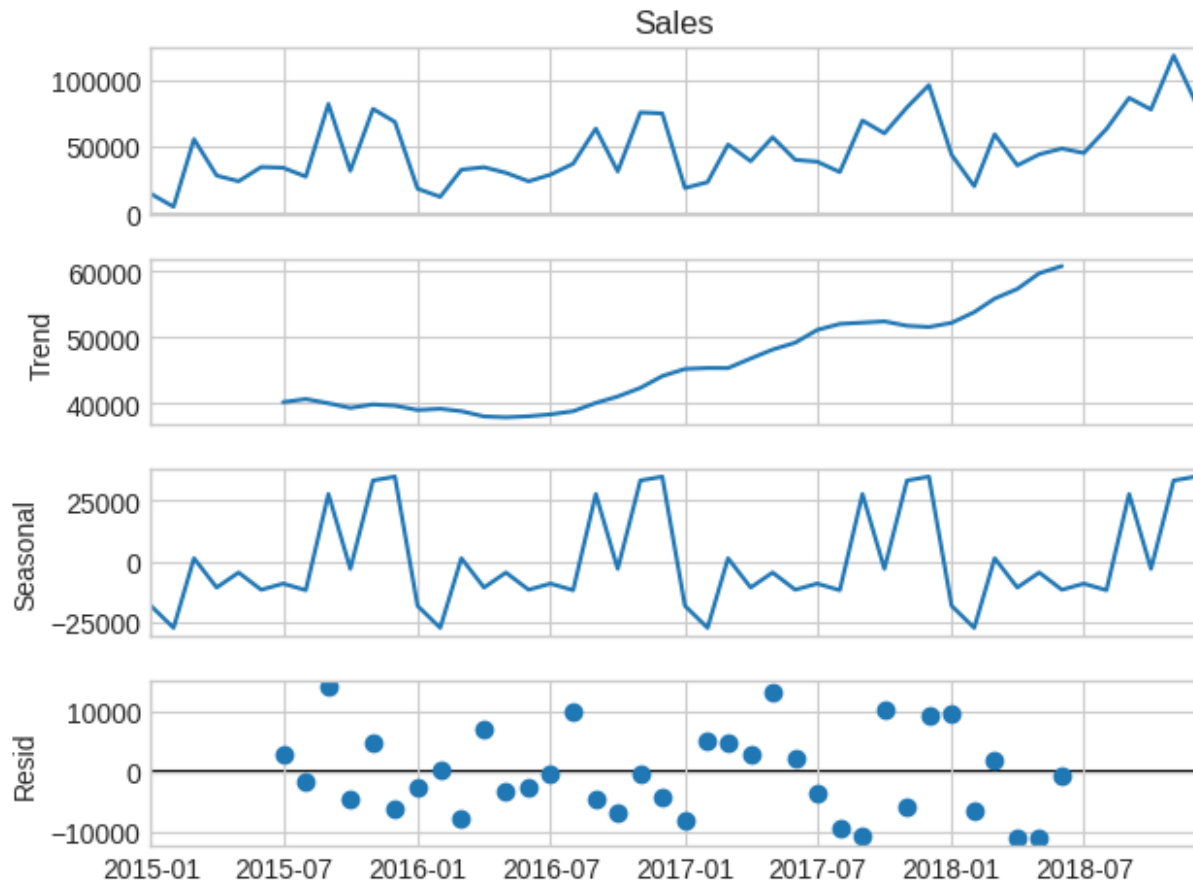


SEASONAL ANALYSIS

```
decomposition = seasonal_decompose(  
    monthly_sales,  
    model='additive',  
    period=12  
)
```

```
fig = decomposition.plot()  
fig.set_size_inches(15, 10)  
plt.show()
```

<Figure size 1500x1000 with 0 Axes>



MONTHLY SEASONAL PATTERN

```
df['Month'] = df['Order Date'].dt.month
```

```
monthly_pattern = (
    df.groupby('Month')['Sales']
    .sum()
)
```

```
monthly_pattern
```

```
Month
```

1	94291.6296
2	59371.1154
3	197573.5872
4	136283.0006
5	154086.7237
6	145837.5233
7	145535.6890
8	157315.9270
9	300103.4117

```

10     199496.2947
11     350161.7110
12     321480.1695
Name: Sales, dtype: float64

plt.figure(figsize=(12,5))

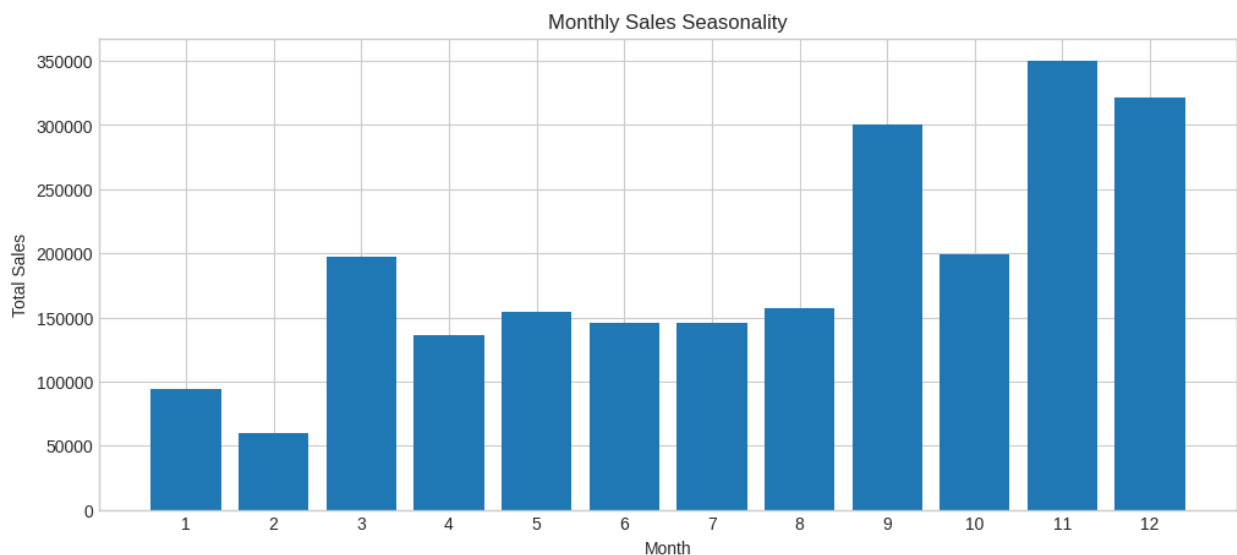
plt.bar(
    monthly_pattern.index,
    monthly_pattern.values
)

plt.title('Monthly Sales Seasonality')
plt.xlabel('Month')
plt.ylabel('Total Sales')

plt.xticks(range(1,13))

plt.show()

```



POTENTIAL HIGH DEMAND PERIODS

```

sales_mean = monthly_sales.mean()
sales_std = monthly_sales.std()

high_sales_threshold = sales_mean + sales_std

potential_promo_months = monthly_sales[
    monthly_sales > high_sales_threshold
]

```

```
print("Potential high-demand periods:")
print(potential_promo_months)
```

Potential high-demand periods:

Order Date

2015-09-01 81623.5268

2015-11-01 77907.6607

2016-11-01 75249.3995

2016-12-01 74543.6012

2017-11-01 79066.4958

2017-12-01 95739.1210

2018-09-01 86152.8880

2018-10-01 77448.1312

2018-11-01 117938.1550

2018-12-01 83030.3888

Name: Sales, dtype: float64

```
plt.figure(figsize=(15,6))
```

```
plt.plot(
    monthly_sales,
    label='Monthly Sales'
)
```

```
plt.axhline(
    high_sales_threshold,
    linestyle='--',
    label='High-Sales Threshold'
)
```

```
plt.scatter(
    potential_promo_months.index,
    potential_promo_months.values,
    s=80,
    label='Potential High-Demand Period'
)
```

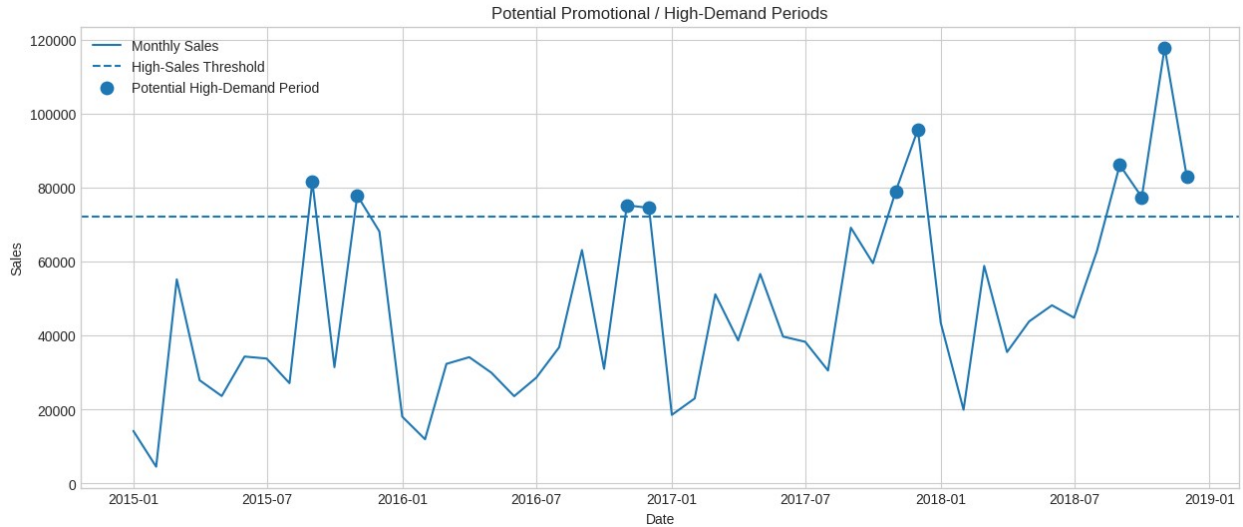
```
plt.title('Potential Promotional / High-Demand Periods')
```

```
plt.xlabel('Date')
```

```
plt.ylabel('Sales')
```

```
plt.legend()
```

```
plt.show()
```



```
adf_result = adfuller(monthly_sales)

print("ADF Statistic:", adf_result[0])
print("p-value:", adf_result[1])

ADF Statistic: -4.416136761430769
p-value: 0.00027791039276670623

sales_diff = monthly_sales.diff().dropna()

adf_result_diff = adfuller(sales_diff)

print("ADF Statistic:", adf_result_diff[0])
print("p-value:", adf_result_diff[1])

ADF Statistic: -8.727061830353268
p-value: 3.2669175472796045e-14
```

TRAIN AND TEST DATASET

```
train = monthly_sales.iloc[:-12]
test = monthly_sales.iloc[-12:]

print("Training data:", len(train))
print("Testing data:", len(test))

Training data: 36
Testing data: 12
```

SARIMA MODEL

```
model = SARIMAX(
    train,
    order=(1,1,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,
    enforce_invertibility=False
)
```

```
model_fit = model.fit()
```

```
print(model_fit.summary())
```

SARIMAX Results

```
=====
=====
Dep. Variable:                Sales    No. Observations:
36
Model:                SARIMAX(1, 1, 1)x(1, 1, 1, 12)    Log Likelihood
-96.483
Date:                Thu, 13 Aug 2026    AIC
202.966
Time:                13:20:07    BIC
203.952
Sample:                01-01-2015    HQIC
200.838
                        - 12-01-2017

Covariance Type:                opg

=====
=====
                        coef    std err          z      P>|z|      [0.025
0.975]
-----
-----
ar.L1                -0.2270      0.644     -0.352      0.725     -1.489
1.035
ma.L1                -0.9915      0.829     -1.196      0.232     -2.617
0.634
ar.S.L12             -0.5174      1.498     -0.345      0.730     -3.454
2.419
ma.S.L12              0.5488      1.757      0.312      0.755     -2.894
3.992
sigma2              1.385e+08    1.44e-08    9.62e+15     0.000     1.38e+08
1.38e+08
=====
=====
```

```

Ljung-Box (L1) (Q):          0.00   Jarque-Bera (JB):
0.41
Prob(Q):                    0.99   Prob(JB):
0.82
Heteroskedasticity (H):      1.26   Skew:
-0.29
Prob(H) (two-sided):        0.85   Kurtosis:
2.14
=====
=====

```

Warnings:

```

[1] Covariance matrix calculated using the outer product of gradients
(complex-step).
[2] Covariance matrix is singular or near-singular, with condition
number 1.67e+32. Standard errors may be unstable.

```

ACTUAL VS PREDICTED FORECAST

```

forecast_result = model_fit.get_forecast(
    steps=len(test)
)

forecast = forecast_result.predicted_mean

confidence_interval = forecast_result.conf_int()

plt.figure(figsize=(15,7))

plt.plot(
    train.index,
    train,
    label='Training Sales'
)

plt.plot(
    test.index,
    test,
    marker='o',
    label='Actual Sales'
)

plt.plot(
    test.index,
    forecast,
    marker='o',
    label='Forecasted Sales'
)

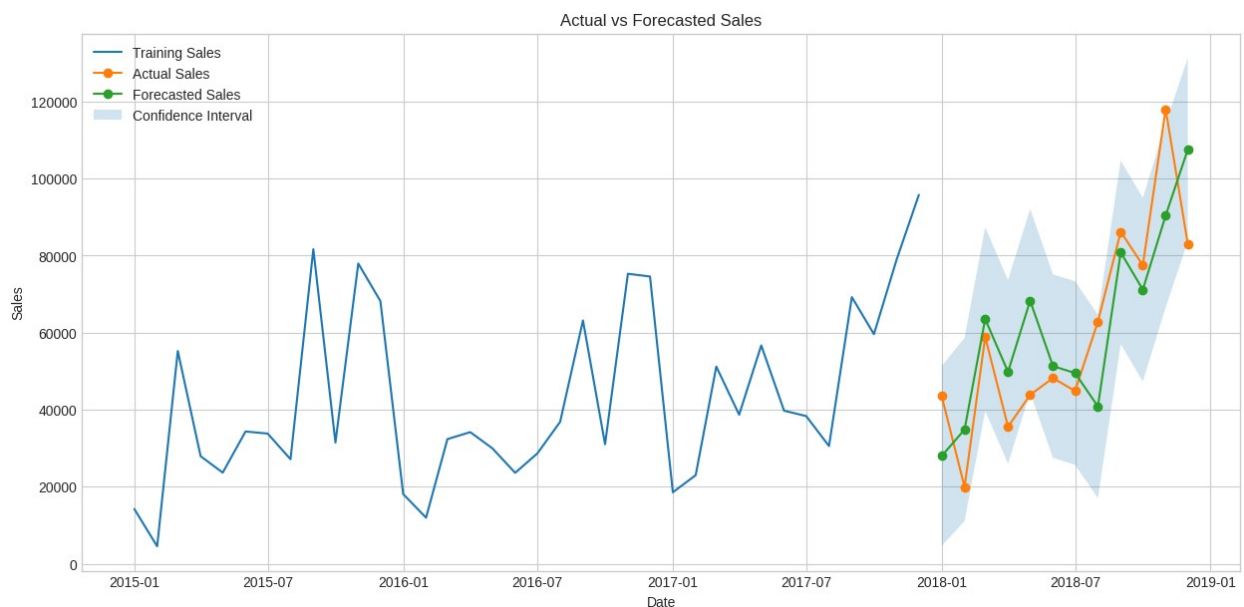
```

```
plt.fill_between(
    test.index,
    confidence_interval.iloc[:,0],
    confidence_interval.iloc[:,1],
    alpha=0.2,
    label='Confidence Interval'
)

plt.title('Actual vs Forecasted Sales')
plt.xlabel('Date')
plt.ylabel('Sales')

plt.legend()

plt.show()
```



EVALUATION METRICS

```
mae = mean_absolute_error(
    test,
    forecast
)
```

```
print("MAE:", mae)
```

```
MAE: 13930.017297227341
```

```
rmse = np.sqrt(
    mean_squared_error(
```

```

        test,
        forecast
    )
)

print("RMSE:", rmse)

mape = np.mean(
    np.abs(
        (test - forecast) / test
    )
) * 100

print("MAPE:", mape)
MAPE: 27.7680896334966

rmse = np.sqrt(
    mean_squared_error(
        test,
        forecast
    )
)

evaluation = pd.DataFrame({
    'Metric': ['MAE', 'RMSE', 'MAPE'],
    'Value': [mae, rmse, mape]
})

evaluation

{"summary": "{\n  \"name\": \"evaluation\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\": \"Metric\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"MAE\",\n          \"RMSE\",\n          \"MAPE\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Value\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 8824.474019464531,\n        \"min\": 27.7680896334966,\n        \"max\": 16394.816726050944,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          13930.017297227341,\n          16394.816726050944,\n          27.7680896334966\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\",\n  \"variable_name\": \"evaluation\"\n}", "type": "dataframe", "variable_name": "evaluation"}

final_model = SARIMAX(
    monthly_sales,
    order=(1,1,1),
    seasonal_order=(1,1,1,12),
    enforce_stationarity=False,

```

```

        enforce_invertibility=False
    )

    final_model_fit = final_model.fit()

    future_result = final_model_fit.get_forecast(
        steps=12
    )

    future_forecast = future_result.predicted_mean
    future_confidence = future_result.conf_int()

```

FITURE SALES FORECAST

```

future_sales = pd.DataFrame({
    'Month': future_forecast.index,
    'Forecast Sales': future_forecast.values,
    'Lower Bound': future_confidence.iloc[:,0].values,
    'Upper Bound': future_confidence.iloc[:,1].values
})

future_sales = future_sales.round(2)

future_sales

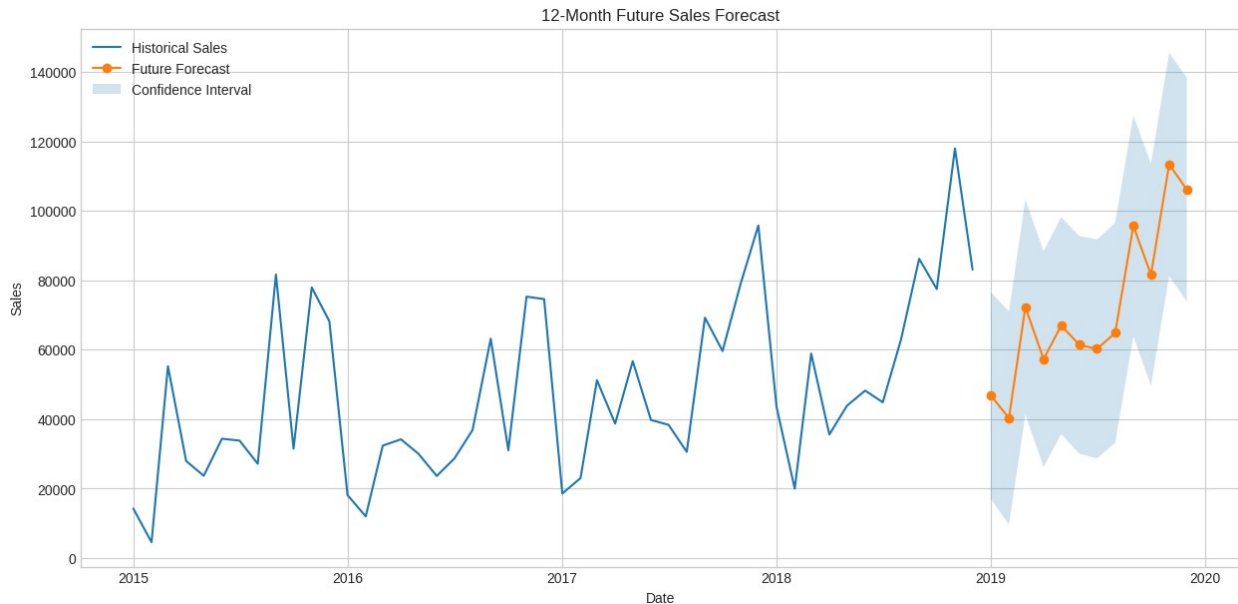
{"summary": "{\n  \"name\": \"future_sales\",\n  \"rows\": 12,\n  \"fields\": [\n    {\n      \"column\": \"Month\",\n      \"properties\": {\n        \"dtype\": \"date\",\n        \"min\": \"2019-01-01 00:00:00\",\n        \"max\": \"2019-12-01 00:00:00\",\n        \"num_unique_values\": 12,\n        \"samples\": [\n          \"2019-11-01 00:00:00\",\n          \"2019-10-01 00:00:00\",\n          \"2019-01-01 00:00:00\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Forecast Sales\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 22785.581735585987,\n        \"min\": 40285.47,\n        \"max\": 113342.88,\n        \"num_unique_values\": 12,\n        \"samples\": [\n          113342.88,\n          81551.12,\n          46782.48\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Lower Bound\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 22240.57170904607,\n        \"min\": 9596.04,\n        \"max\": 81253.76,\n        \"num_unique_values\": 12,\n        \"samples\": [\n          81253.76,\n          49604.93,\n          16992.67\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Upper Bound\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":

```

```

23337.95505503757,\n          \"min\": 70974.9,\n          \"max\":  
145432.0,\n          \"num_unique_values\": 12,\n          \"samples\": [\n            145432.0,\n            113497.31,\n            76572.28\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }\n      ],\n      \"type\": \"dataframe\", \"variable_name\": \"future_sales\"}\n    )\n\nplt.figure(figsize=(15,7))\n\nplt.plot(\n    monthly_sales.index,\n    monthly_sales,\n    label='Historical Sales'\n)\n\nplt.plot(\n    future_forecast.index,\n    future_forecast,\n    marker='o',\n    label='Future Forecast'\n)\n\nplt.fill_between(\n    future_forecast.index,\n    future_confidence.iloc[:,0],\n    future_confidence.iloc[:,1],\n    alpha=0.2,\n    label='Confidence Interval'\n)\n\nplt.title('12-Month Future Sales Forecast')\nplt.xlabel('Date')\nplt.ylabel('Sales')\n\nplt.legend()\n\nplt.show()

```



```
future_sales.to_csv(  
    '/content/sales_forecast_12_months.csv',  
    index=False  
)  
  
evaluation.to_csv(  
    '/content/model_evaluation.csv',  
    index=False  
)  
  
print("Files saved successfully.")  
Files saved successfully.
```

CONCLUSION

The SARIMA model successfully forecasted future sales by capturing historical trends and seasonal patterns. The results demonstrate how time series forecasting can support better inventory planning, sales strategy, and data-driven business decisions.